

ESN 11: Co-design labs

Kamel ABDELOUAHAB

kamel@sma-rty.com



The rules



1. 3 x 4 hours
2. Two parts lab :
 - a. Led chaser with IOs
 - b. 7 Segment counter handling
3. **No internet copy-pasta !**

Evaluation ...



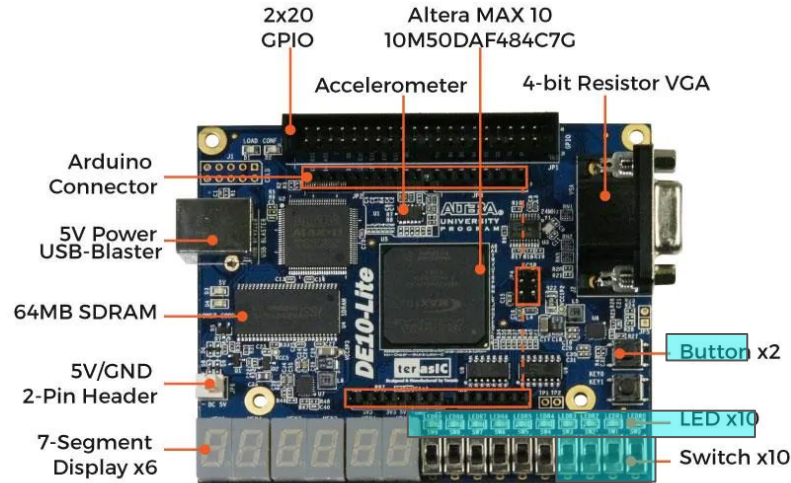
You will be evaluated based on your lab report + Quartus project

- Both have to be sent to kamel@sma-rtty.com
- **HARD DEADLINE: 22/10/2021@13:00**
- Report content:
 - a. Introduction (material, tools ...)
 - b. Lab1: global system overview, progress, results (take photos if needed)
 - c. Lab2: global system overview, progress, results (take photos if needed)
 - d. Conclusion: What have you learned, comment progress, pitfalls and solutions

Lab 1: “*Let there be light*”



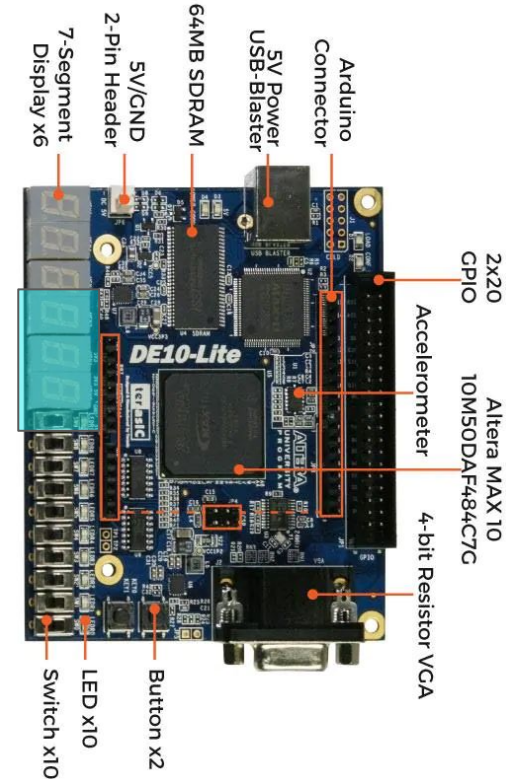
1. Create a basic Nios II system
2. Add blocks and software to light a LED with a C program
3. Implement an X LEDs Chaser in software
4. Add a push button to trigger the chaser
5. Use switch buttons to set the speed on 4 bits



Lab2: “Let’s count stuff”



1. Create a basic Nios II system
2. Implement a 1-digit decimal counter and display it on the 7-seg display
 - a. Write your software counter
 - b. Write you bin_to_7seg.vhd component
 - c. Wire your system
3. Implement a 3-digit counter counter and display it on the 7-seg display
 - a. Write your software counter
 - b. Write a software or hardware implementation of the double dabble algorithm (motivate your choice on your report)
 - c. Wire your system !





Algorithm [\[edit \]](#)

The algorithm operates as follows:

Suppose the original number to be converted is stored in a [register](#) that is n bits wide. Reserve a scratch space wide enough to hold both the original number and its BCD representation; $n + 4 \times \text{ceil}(n/3)$ bits will be enough. It takes a maximum of 4 bits in binary to store each decimal digit.

Then partition the scratch space into BCD digits (on the left) and the original register (on the right). For example, if the original number to be converted is eight bits wide, the scratch space would be partitioned as follows:

100s	Tens	Ones	Original
0010	0100	0011	11110011

The diagram above shows the binary representation of 243_{10} in the original register, and the BCD representation of 243 on the left.

The scratch space is initialized to all zeros, and then the value to be converted is copied into the "original register" space on the right.

0000	0000	0000	11110011
------	------	------	----------

The algorithm then iterates n times. On each iteration, any BCD digit which is at least 5 (0101 in binary) is incremented by 3 (0011); then the entire scratch space is left-shifted one bit. The increment ensures that a value of 5, incremented and left-shifted, becomes 16 (10000), thus correctly "carrying" into the next BCD digit.

Essentially, the algorithm operates by doubling the BCD value on the left each iteration and adding either one or zero according to the original bit pattern. Shifting left accomplishes both tasks simultaneously. If any digit is five or above, three is added to ensure the value "carries" in base 10.

The double-dabble algorithm, performed on the value 243_{10} , looks like this:

0000	0000	0000	11110011	Initialization
0000	0000	0001	11100110	Shift
0000	0000	0011	11001100	Shift
0000	0000	0111	10011000	Shift
0000	0000	1010	10011000	Add 3 to ONES, since it was 7
0000	0001	0101	00110000	Shift
0000	0001	1000	00110000	Add 3 to ONES, since it was 5
0000	0011	0000	01100000	Shift
0000	0110	0000	11000000	Shift
0000	1001	0000	11000000	Add 3 to TENS, since it was 6
0001	0010	0001	10000000	Shift
0010	0100	0011	00000000	Shift
2	4	3		
				BCD

Counter with timers



1. The timer generates the interrupt at each second
2. The ISR corresponding to interrupt increments 4 bits counter
3. The value of the counter is sent to a VHDL instance
4. Design a binary to 7-seg decoder

